Phoenix's generator to help your application feel a bit cleaner. The generated code for this module uses a bit of dynamic programming, but we can see that the router function just uses and imports other modules:

phone_app/lib/phone_app_web.ex

```
defmodule PhoneAppWeb do
  # Phoenix uses dynamic programming to clean up the `use` function
  defmacro __using__(which) when is_atom(which) do
    apply(__MODULE__, which, [])
  end

  def router do
    quote do
      use Phoenix.Router, helpers: false

      # Import common connection and controller functions to use in pipelines
      import Plug.Conn
      import Phoenix.Controller
      import Phoenix.LiveView.Router
    end
  end
end
```

You'll see this pattern in most modules that use Phoenix modules—it keeps your application code cleaner.

Let's go back to our router. The pipeline function defines a set of plugs that a particular request will flow through. This lets you conditionally invoke middleware based on the route being called.

A pipeline is referenced with the pipe_through/1 function. If necessary, you can pipe through multiple pipelines, such as pipe_through [:api, :authenticated].

The scope function is used to group a collection of routes together. The way that you scope your requests will depend on your application but is often done based on different authentication needs or URL grouping.

The line get "/", PageController, :home defines a handler for requests to GET /. A request will be sent to the PhoneAppWeb.PageController.home/2 function. There are other functions corresponding to different HTTP verbs, such as post, put, and delete.

In addition to individual HTTP verbs, you can use resources/4[14] to define a full suite of endpoints in one line. For example, the following line of code would produce the noted routes:

```
resources "/widgets", WidgetController

# The following routes are generated, viewable with `mix phx.routes`
```

14. https://hexdocs.pm/phoenix/Phoenix.Router.html#resources/4